A Fully Offline Android Prototype for Certificate OCR and Structured Field Extraction

Project N152 Research Team

Abstract - *This paper presents a fully offline Android prototype that converts camera-captured certificates into readable text, preserves basic document layout, extracts five certificate fields, and stores the result with its source image. The pipeline combines classical image processing with compact recognition and language models. OpenCV performs border detection, perspective correction, quality checks, contrast enhancement, and sharpening. Google ML Kit handles printed text, while a PP-OCRv5 mobile recognizer executed through ONNX Runtime provides a handwriting path. Geometry-aware formatting and conservative token correction reduce common OCR artifacts. Certificate name, issue date, port of issue, issuing authority, and expiry date are recovered through label-aware rules with MobileBERT question answering as a fallback. A corpus supplied for adaptation contained 108 PDF files and 351 pages, of which 295 page images were unique. Bootstrap OCR produced 20,106 line crops and 18,166 high-confidence pseudo-labels. A 4.37 million parameter BERT model fine-tuned on weak field labels reached 35.3 percent validation exact match and 23.5 percent test exact match, so it was retained as an experimental asset rather than used as the sole extractor. The implemented hybrid is functional on an Android 15 phone, but a manually labeled end-to-end evaluation is still required before deployment claims can be made.*

Index Terms - *Android, certificate digitization, document image processing, offline OCR, PP-OCRv5, MobileBERT, information extraction*

I. INTRODUCTION

Certificates are often exchanged as photographs, scanned PDFs, or paper records. Turning them into searchable data is difficult when images contain perspective distortion, shadows, low contrast, mixed typography, stamps, or handwriting. Cloud document services can address some of these problems, but they also introduce network dependence and expose potentially sensitive records to an external service. An offline mobile workflow is useful when connectivity is unreliable, privacy is important, or users need immediate results at the point of capture.

Mobile OCR has been studied since early Android devices. OCRdroid combined on-device preprocessing with server recognition, which showed the value of guided capture but retained a backend dependency [2]. Modern lightweight OCR systems move more work to the device. PP-OCR demonstrated that compact detectors and recognizers can provide practical accuracy with small models [3], and later versions improved that balance through better backbones, training objectives, and data strategies [4], [5]. PP-OCRv5 continues this direction with a specialized five million parameter system intended to compete with much larger vision-language models on OCR tasks [6].

The Project N152 prototype investigates how far an Android application can go without a network connection. It does not attempt to replace a production document intelligence platform. Instead, it integrates capture guidance, geometric correction, printed and handwritten recognition, readable text reconstruction, certificate field extraction, local organization, and document question answering into one testable application.

The work makes four practical contributions:

- An offline Android pipeline that joins classical document rectification with separate printed and handwriting recognition paths.
- A geometry-aware text reconstruction stage that retains line breaks and approximate indentation instead of returning a flat OCR string.
- A hybrid certificate extractor for certificate name, issue date, port of issue, issuing authority, and expiry date, using deterministic patterns before a compact question-answering fallback.
- A reproducible adaptation pipeline over the supplied certificate corpus, including duplicate removal, document-level splitting, pseudo-label generation, tiny BERT fine-tuning, ONNX export, and an explicit report of weak results.

II. RELATED WORK

A. OCR on Documents and Mobile Devices

Tesseract established a widely used open OCR engine and described an adaptive recognition architecture suitable for document images [1]. Camera capture creates additional problems that flatbed scans usually avoid. OCRdroid identified focus loss, illumination variation, skew, and text misalignment as practical mobile concerns [2]. The current prototype addresses the same capture conditions locally through contour analysis, perspective correction, brightness checks, blur estimation, and contrast enhancement.

Text detection research also informs mobile preprocessing. CRAFT links character regions through learned affinity scores and is effective on irregular text shapes [14]. Differentiable Binarization integrates adaptive thresholding into a segmentation detector and showed a useful speed-accuracy tradeoff with lightweight backbones [15]. These models are not embedded in the prototype because the primary target is a bounded document rather than arbitrary scene text. Their results nevertheless support a future learned detector when contour-based border discovery fails.

B. Lightweight and Transformer OCR

PP-OCR is directly relevant to constrained deployment because it treats efficiency as a first-class design goal [3]. PP-OCRv2 and PP-OCRv3 improve the detector and recognizer while preserving a lightweight profile [4], [5]. The prototype uses a PP-OCRv5 English mobile recognition model for cropped handwriting lines. The model is executed through ONNX Runtime to avoid a server call and falls back to ML Kit output when its confidence is low.

Transformer OCR offers a different design point. TrOCR combines a pretrained image transformer with a text transformer and reports strong printed, handwritten, and scene text recognition [7]. Donut removes the external OCR stage and learns document understanding directly from images [10]. DeepSeek-OCR and Unlimited OCR extend

visual compression and long-document processing, respectively [16], [17]. These models are relevant research targets, but their scale and runtime requirements are less suitable for the current phone-first prototype than a modular lightweight pipeline.

C. Document Understanding and Mobile NLP

LayoutLM combines text and two-dimensional layout information for visually rich documents [8]. LayoutLMv3 unifies text and image masking during pretraining [9]. Both motivate preserving OCR geometry because reading order and spatial relationships can matter as much as the recognized tokens. Donut instead avoids OCR error propagation through direct image-to-sequence learning [10]. The prototype follows the OCR-based route because it supports independent debugging, compact mobile components, and direct user editing.

BERT established bidirectional transformer pretraining for language understanding [11]. MobileBERT compresses this capability for resource-limited devices and reports a substantially smaller and faster model than BERT Base [12]. The prototype uses a TensorFlow Lite MobileBERT question-answering model for local document questions and as a fallback in certificate field extraction. Public benchmarks such as FUNSD and CORD provide form and receipt annotations [13], [18], but neither matches the target certificate fields exactly. This mismatch motivates domain adaptation on the supplied corpus.

III. SYSTEM DESIGN

Figure 1 shows the implemented workflow. The user can capture a page with the camera or select an existing image. All subsequent stages run locally. The source image is retained so that OCR output can be checked against the original document.

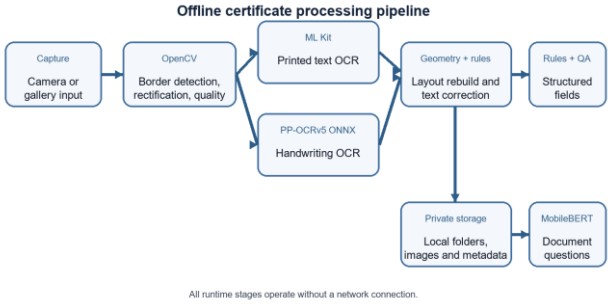


Fig. 1. Implemented offline processing pipeline.

A. Capture, Detection, and Rectification

The analysis image is limited to a maximum dimension of 1,600 pixels to control memory use. The app converts the image to grayscale, applies Gaussian smoothing, runs Canny edge detection with thresholds of 55 and 170, and closes broken edges with a 5 by 5 morphological kernel. Contours are approximated as polygons. A convex quadrilateral covering at least 15 percent of the image is considered a document candidate. Ordered corner points define a perspective transform that produces a top-down page image.

Three low-cost checks warn about unsuitable capture conditions. Mean grayscale intensity below 55 indicates a dark image, while a value above 220 indicates overexposure. Laplacian variance below 65 indicates a soft or blurred image. These thresholds are heuristics, not learned quality scores, and remain candidates for device-specific calibration.

B. Image Enhancement

The rectified image is enlarged only when necessary and is capped at a 3,200 pixel OCR dimension. Contrast-limited adaptive histogram equalization uses a clip limit of 2.2 and an 8 by 8 tile grid. A 3 pixel median filter reduces isolated noise. The black-and-white mode applies adaptive thresholding with a block size of 35 and a constant of 11. The color and grayscale modes use unsharp masking with a Gaussian sigma between 1.05 and 1.35 and an amount between 1.32 and 1.50. An 18 pixel white border prevents edge characters from touching the crop boundary. OpenCV supplies these deterministic operations [19].

C. Printed and Handwritten Recognition

Printed text is recognized with Google ML Kit Text Recognition v2, version 16.0.1 in the prototype [20]. ML Kit returns blocks, lines, elements, and bounding boxes that are reused during layout reconstruction. The handwriting mode first obtains candidate line regions and then sends normalized line crops to a PP-OCRv5 English mobile recognizer. Inputs use a height of 48 pixels and a width of up to 2,048 pixels. Connectionist Temporal Classification decoding converts the model output to text. Results below a confidence threshold of 0.20 are replaced with the corresponding ML Kit line to avoid confidently displaying unstable handwriting output.

D. Layout Reconstruction and Correction

OCR blocks are ordered from top to bottom and then from left to right. Lines inside each block are sorted by vertical position. The formatter estimates a median character width and converts relative horizontal offsets into zero to six leading spaces. This approximation cannot reproduce complex tables, but it keeps headings, paragraphs, and simple indentation more readable than a plain concatenated string.

A conservative post-correction stage targets short OCR confusions without rewriting the document. It considers common digit-letter substitutions such as 0 and o, 1 and l, 5 and s, 6 and g, and 8 and b. Candidate words are compared with a small local dictionary using Levenshtein distance. Uppercase identifiers and mixed alphanumeric codes are protected because aggressive language correction can corrupt certificate numbers, ports, and abbreviations.

E. Structured Extraction and Local Use

The certificate extractor searches OCR lines for field labels and nearby values. Date expressions are normalized only when the surrounding label indicates issue or expiry. Authority and port candidates are ranked by label proximity. A MobileBERT question-answering fallback asks targeted questions over the OCR text when deterministic extraction does not produce a field. The same local model supports general document questions by retrieving overlapping 1,200 character chunks with a 180 character overlap before answer extraction. Documents, preview images, folders, notes, highlights, bookmarks, source type, and recognition mode are stored in app-private files.

IV. DATASET AND MODEL ADAPTATION

A. Corpus Audit and Split

The supplied archive was described as containing 114 certificates. Direct inspection found 108 PDF files with 351 pages. Page image hashing identified 295 unique pages and 56 exact duplicates. Only 130 pages contained an extractable text layer, while 221 were image-only. To reduce leakage, unique pages were grouped by source document before splitting. The resulting train, validation, and test sets contained 206, 56, and 33 unique pages, respectively.

TABLE I. CERTIFICATE CORPUS AND BOOTSTRAP LABELS

Item	Count
PDF files	108
Total pages	351
Unique pages	295
Exact duplicate pages	56
Text-layer pages	130
Image-only pages	221
Train / validation / test pages	206 / 56 / 33
OCR line crops	20,106
High-confidence pseudo-labels	18,166
Review candidates	1,940

B. Recognition Bootstrap

A PP-OCRv6 small pipeline generated line crops and provisional transcriptions from the 295 unique pages. Confidence at or above 0.85 marked a pseudo-label as high-confidence. This produced 20,106 line crops, including 18,166 high-confidence labels and 1,940 review candidates. The train split contained 16,139 lines, the validation split 2,523 lines, and the test split 1,444 lines. These values describe automatically generated training material, not verified ground truth. They are therefore suitable for bootstrapping and triage but not for reporting final character accuracy.

C. Certificate Field QA Data

A second preparation stage built weak question-answering examples for the five target fields. Files whose names indicated drawings, layouts, seating plans, schematics, specifications, manuals, data sheets, or calculations were excluded to keep the task focused on certificates. The generator created 129 training, 35 validation, and 19 test examples. After token-window alignment, 123, 34, and 17 examples remained usable. The labels came from text-layer and heuristic extraction, so they may contain missing or shifted answer spans.

D. Tiny BERT Adaptation

The experimental language model started from mightyneighbor/bert-tiny-finetuned-squad with approximately 4.37 million parameters. It was fine-tuned for 30 CPU epochs and exported to ONNX as a 17.5 MB asset. Strict exact-span match was used because a field must be copied precisely from the certificate context. The best validation exact match was 0.3529 and the test exact match was 0.2353. This result is too low for autonomous field extraction. The asset is retained for continued experiments, while the app uses deterministic extraction and the stronger bundled MobileBERT model as its operational path.

TABLE II. EXPERIMENTAL TINY BERT RESULT

Measure	Result
Aligned train / validation / test	123 / 34 / 17
Training epochs	30
Best validation exact match	35.3%
Test exact match	23.5%
Exported ONNX size	17.5 MB
Deployment decision	Experimental only

V. PROTOTYPE IMPLEMENTATION

The application is written in Kotlin with Jetpack Compose and targets Android API level 36 with a minimum level of 24. The main runtime dependencies and their roles are listed in Table III. The PP-OCRv5 English recognition asset is about 7.96 MB. The MobileBERT question-answering asset is about 100.46 MB with a 0.23 MB vocabulary. The assembled debug APK is 471,632,775 bytes, or about 449.8 MiB. The APK size is acceptable for a demonstration but excessive for distribution and is mainly driven by native runtimes and embedded models.

TABLE III. IMPLEMENTED MOBILE COMPONENTS

Component	Version / role
Kotlin + Compose	Android user interface
OpenCV	4.13.0, document image processing
ML Kit Text Recognition	16.0.1, printed OCR and line geometry
ONNX Runtime Android	1.24.1, PP-OCRv5 execution
TensorFlow Lite Task Text	0.4.4, MobileBERT QA
Android Image Cropper	4.7.0, manual correction
Coil Compose	2.7.0, local preview display

The debug build passed its unit tests, including a certificate field extraction sample, and installed through Android Debug Bridge on an OPPO CPH2665 running Android 15 with an arm64-v8a processor. This verifies buildability, packaging, installation, and basic functional flow. It does not establish OCR accuracy, field-level F1, latency, memory consumption, or battery cost.

VI. EVALUATION AND RESULTS

A. Functional Verification

The prototype was evaluated at three levels. First, build and unit checks confirmed that the Kotlin code, native libraries, and model assets package successfully. Second, device installation confirmed compatibility with the connected Android 15 handset. Third, corpus processing confirmed that the data preparation scripts can extract pages, remove exact duplicates, create document-level splits, generate line crops, produce weak field examples, fine-tune a compact QA model, and export it to ONNX.

The practical OCR path improved markedly after document rectification and enhancement were introduced, based on iterative user testing during prototype development. That observation motivated the current pipeline, but it is qualitative and is not presented as a benchmark. No paired raw-versus-enhanced evaluation set was recorded, so a numerical preprocessing gain cannot be claimed.

B. Interpretation of the Fine-Tuning Result

The tiny BERT result exposes the main data problem. Seventeen aligned test samples are not enough for a stable estimate across five fields and multiple certificate layouts. More importantly, weak answer spans can teach the model the errors made by the extractor that created them. A model can therefore optimize the training objective while failing to learn reliable field boundaries. The low exact match supports the decision to keep deterministic label matching in front of the language model.

The result also suggests that field extraction should not be treated as ordinary question answering alone. Issue and expiry dates may share the same syntax, and their meaning depends on nearby labels and layout. Ports and authorities can be short entities that also appear elsewhere in a document. A future model should use token coordinates or image features, as in layout-aware document models [8], [9], while still fitting the mobile resource budget.

VII. DISCUSSION

A. Why the Hybrid Design Is Appropriate

A modular hybrid is less elegant than a single end-to-end model, but it matches the constraints of a prototype. Every stage can be inspected: the user can compare the source image with the rectified image, see OCR text, correct the crop, and identify which structured fields were found. Classical preprocessing has predictable cost and does not need training data. Compact OCR models handle recognition, while deterministic rules protect high-value identifiers from language-model rewriting. MobileBERT is used only where context is needed.

The architecture also degrades gracefully. If border detection fails, manual cropping remains available. If handwriting confidence is low, the system can use the printed OCR line. If a field cannot be extracted, the document and raw text are still stored. This behavior is preferable to silently generating plausible but unsupported values.

B. Limitations

The strongest limitation is the absence of manually verified ground truth for end-to-end evaluation. The corpus contains exact duplicate pages, mixed document types, and a large image-only subset. Pseudo-labels are useful for scale but cannot measure the errors of the model that produced them. The field QA test set contains only 17 aligned examples. No character error rate, word error rate, field precision, recall, or F1 score has yet been measured.

The runtime has additional constraints. The current correction dictionary is small and English-oriented. Approximate indentation does not reconstruct complex tables. Contour-based page detection may fail on borderless pages or backgrounds with stronger rectangular edges. The 449.8 MiB debug APK is too large for ordinary distribution. Latency, thermal behavior, peak memory, and battery use have not been profiled across devices. The application is therefore a working research prototype, not a production-ready certificate verifier.

C. Next Evaluation

The next useful step is a small but carefully labeled benchmark rather than more weak training data. A stratified set should cover printed certificates, handwriting, stamps, skew, uneven light, and multiple issuing formats. Each page needs a verified transcript, five field labels, and document corners. The app can then report border intersection-over-union, character and word error rates, per-field precision, recall, and F1, plus median and 95th percentile latency. Only after this baseline exists should a learned document detector, layout-aware extractor, quantized transformer, or OCR-free model replace a working component.

VIII. CONCLUSION

Project N152 demonstrates that a useful certificate digitization workflow can run entirely on an Android phone. The prototype combines OpenCV rectification and enhancement, ML Kit printed OCR, PP-OCRv5 handwriting recognition through ONNX Runtime, geometry-aware formatting, conservative correction, hybrid field extraction, local storage, and MobileBERT document questions. It also provides a reproducible path from a supplied PDF archive to deduplicated pages, OCR line crops, weak field labels, compact model fine-tuning, and mobile export.

The experiments do not support a claim of production accuracy. The custom tiny BERT model achieved only 23.5 percent strict exact match on the small test set, and the complete application has not yet been measured against manually labeled pages. This negative result is valuable because it identifies the next bottleneck: trustworthy annotation and field-level evaluation. The current hybrid should remain the baseline while that evidence is collected.

REFERENCES

[1] R. Smith, "An overview of the Tesseract OCR engine," in Proc. 9th Int. Conf. Document Analysis and Recognition, 2007, pp. 629-633, doi: 10.1109/ICDAR.2007.4376991.

[2] M. Zhang, A. Joshi, R. Kadmawala, K. Dantu, S. Poduri, and G. S. Sukhatme, "OCRdroid: A framework to digitize text using mobile phones," in Mobile Computing, Applications, and Services, 2009, pp. 273-292, doi: 10.1007/978-3-642-12607-9_18.

[3] Y. Du et al., "PP-OCR: A practical ultra lightweight OCR system," arXiv:2009.09941, 2020.

[4] Y. Du et al., "PP-OCRv2: Bag of tricks for ultra lightweight OCR system," arXiv:2109.03144, 2021.

[5] C. Li et al., "PP-OCRv3: More attempts for the improvement of ultra lightweight OCR system," arXiv:2206.03001, 2022.

[6] C. Cui et al., "PP-OCRv5: A specialized 5M-parameter model rivaling billion-parameter vision-language models on OCR tasks," arXiv:2603.24373, 2026.

[7] M. Li et al., "TrOCR: Transformer-based optical character recognition with pre-trained models," Proc. AAAI Conf. Artificial Intelligence, vol. 37, no. 11, pp. 13094-13102, 2023, doi: 10.1609/aaai.v37i11.26538.

[8] Y. Xu et al., "LayoutLM: Pre-training of text and layout for document image understanding," in Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2020, pp. 1192-1200, doi: 10.1145/3394486.3403172.

[9] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: Pre-training for document AI with unified text and image masking," in Proc. 30th ACM Int. Conf. Multimedia, 2022, pp. 4083-4091, doi: 10.1145/3503161.3548112.

[10] G. Kim et al., "OCR-free document understanding transformer," in Computer Vision - ECCV 2022, vol. 13688, pp. 498-517, 2022, doi: 10.1007/978-3-031-19815-1_29.

[11] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. NAACL-HLT, 2019, pp. 4171-4186, doi: 10.18653/v1/N19-1423.

[12] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, and D. Zhou, "MobileBERT: A compact task-agnostic BERT for resource-limited devices," in Proc. 58th Annu. Meeting Association for Computational Linguistics, 2020, pp. 2158-2170, doi: 10.18653/v1/2020.acl-main.195.

[13] G. Jaume, H. K. Ekenel, and J.-P. Thiran, "FUNSD: A dataset for form understanding in noisy scanned documents," in Proc. ICDAR Workshops, 2019, pp. 1-6, doi: 10.1109/ICDARW.2019.10029.

[14] Y. Baek, B. Lee, D. Han, S. Yun, and H. Lee, "Character region awareness for text detection," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition, 2019, pp. 9365-9374.

[15] M. Liao, Z. Wan, C. Yao, K. Chen, and X. Bai, "Real-time scene text detection with differentiable binarization," Proc. AAAI Conf. Artificial Intelligence, vol. 34, no. 7, pp. 11474-11481, 2020, doi: 10.1609/aaai.v34i07.6812.

[16] H. Wei, Y. Sun, and Y. Li, "DeepSeek-OCR: Contexts optical compression," arXiv:2510.18234, 2025.

[17] Y. Yin et al., "Unlimited OCR works," arXiv:2606.23050, 2026.

[18] S. Park et al., "CORD: A consolidated receipt dataset for post-OCR parsing," in Workshop on Document Intelligence at NeurIPS, 2019.

[19] G. Bradski, "The OpenCV library," Dr. Dobb's Journal of Software Tools, 2000.

[20] Google, "Text recognition v2," ML Kit for Android, 2026. [Online]. Available: https://developers.google.com/ml-kit/vision/text-recognition/v2/android